UNIT 07 • TI.OO.07.2026

MENGIMPLEMENTASIKAN PEMROGRAMAN BERORIENTASI OBJEK (OOP)

Pengembang	Aditya Ristanto Mukti	**Proyek**	Website Resmi & Profil Industri PT Global Philsonic Technologies
Deskripsi Unit	Dokumentasi penerapan prinsip Object-Oriented Programming (OOP) pada sistem, meliputi perancangan Class ES6, instansiasi Object DTO, Encapsulation pada enkripsi kata sandi, Inheritance pada kelas ApiError, Polimorfisme komponen UI adaptif, serta standarisasi Package dan API Contract.		

I. MATRIKS PEMENUHAN SUB-TITLE OUTLINE

No	Sub-Title Outline	Berkas Implementasi	Deskripsi Teknis Hasil Pengerjaan
1	**Penerapan Class & Inheritance**	`backend/src/utils/ApiError.js`	Perancangan kelas kustom ApiError yang mewarisi (extends) kelas bawaan JavaScript Error.
2	**Object & Encapsulation**	`backend/src/controllers/admin/authController.js`	Enkapsulasi password via Bcrypt hashing dan pemodelan objek respons DTO tanpa properti sensitif.
3	**Polymorphism, Interface & Package**	`frontend/src/components/ui/InteractiveHoverButton.jsx` & `Package Modules`	Komponen UI polimorfis adaptif (vs) dan modularitas CommonJS/ESM.

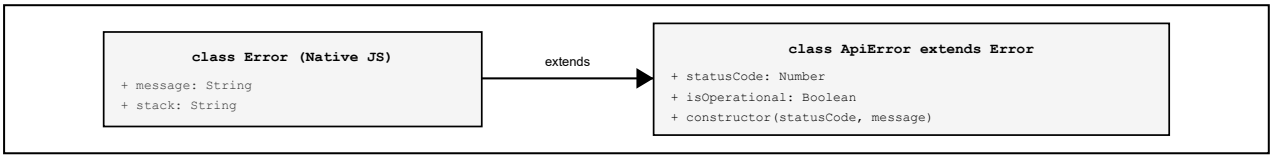
II. PEMBAHASAN TEKNIS INTI & BUKTI KONKRET

7.1 & 7.4. Penerapan Class ES6 & Pewarisan (Inheritance)

Lokasi Berkas: `backend/src/utils/ApiError.js`

Pewarisan kelas diterapkan pada berkas `backend/src/utils/ApiError.js` untuk menciptakan struktur error terkelola yang mempertahankan stack trace bawaan engine V8:

DIAGRAM ALUR / ARSITEKTUR:



Bukti Implementasi Kode Sumber (Visual Studio Code):

backend/src/utils/ApiError.js — VS CODE • UTF-8

```
// Penerapan Class & Inheritance: Mewarisi Kelas Induk 'Error'
class ApiError extends Error {
  constructor(statusCode, message, isOperational = true, stack = '') {
    super(message); // Memanggil konstruktor kelas induk
    this.statusCode = statusCode;
    this.isOperational = isOperational;
    this.status = `${statusCode}`.startsWith('4') ? 'fail' : 'error';

    if (stack) {
      this.stack = stack;
    } else {
      Error.captureStackTrace(this, this.constructor);
    }
  }
}

module.exports = ApiError;
```

7.2 & 7.3. Pemodelan Objek DTO & Enkapsulasi Kredensial Pengguna

Lokasi Berkas: `backend/src/controllers/admin/authController.js`

Enkapsulasi diterapkan untuk menyembunyikan detail sensitif kata sandi melalui hashing Blowfish (Bcrypt) satu arah dan membuang password dari objek transfer data (DTO):

Bukti Implementasi Kode Sumber (Visual Studio Code):

backend/src/controllers/admin/authController.js — VS CODE • UTF-8

```
// Enkapsulasi: Menyembunyikan Sandi Plaintext dengan Bcrypt Hash & DTO Bersih
const login = async (req, res, next) => {
  try {
    const { identity, password } = req.body;

    const [rows] = await pool.execute(
      'SELECT id, name, username, email, password, role, status FROM users WHERE username = ? OR email = ? LIMIT 1',
      [identity, identity]
    );

    if (rows.length === 0) {
      return error(res, 'Kredensial login tidak ditemukan (Username/Email salah).', 401);
    }

    const user = rows[0];
    if (user.status !== 'active') {
      return error(res, 'Akun Anda dalam status nonaktif. Silakan hubungi Administrator.', 403);
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      return error(res, 'Password yang Anda masukkan salah.', 401);
    }

    // Generate JWT Token (Payload DTO Bersih tanpa Properti Password)
    const payload = {
      id: user.id,
      name: user.name,
      username: user.username,
      email: user.email,
      role: user.role,
    };

    const token = jwt.sign(
      payload,
      process.env.JWT_SECRET || 'fallback_secret_key_change_in_production',
      { expiresIn: process.env.JWT_EXPIRES_IN || '8h' }
    );

    await logActivity(user.id, 'LOGIN', 'auth', `User '${user.username}' berhasil login ke sistem`);

    return success(res, 'Login berhasil.', { token, user: payload });
  } catch (err) {
    next(err);
  }
};
```

7.5 & 7.6. Polimorfisme Komponen UI & Adaptasi Antarmuka Interface

Lokasi Berkas: `frontend/src/components/ui/InteractiveHoverButton.jsx`

Polimorfisme antarmuka diterapkan pada `InteractiveHoverButton.jsx` yang secara dinamis menyesuaikan elemen HTML yang dirender (menjadi tag `<Link>` bila menerima prop navigasi `to`, atau tag `<button>` bila berupa aksi event biasa):

Bukti Implementasi Kode Sumber (Visual Studio Code):

frontend/src/components/ui/InteractiveHoverButton.jsx — VS CODE • UTF-8

```jsx
// Polimorfisme Komponen UI: Adaptasi Elemen Render (<Link> vs <button>) Berdasarkan Props
export default function InteractiveHoverButton({
  children,
  text,
  to,
  onClick,
  className = '',
  variant = 'light',
  ...props
}) {
  const content = children || text;
  const innerContent = (
    <>
      <div className="ihb-content-rest">
        <span className="ihb-dot" aria-hidden="true" />
        <span className="ihb-text-rest">{content}</span>
      </div>
      <div className="ihb-content-hover" aria-hidden="true">
        <span className="ihb-text-hover">{content}</span>
      </div>
    </>
  );

  const combinedClass = `interactive-hover-btn interactive-hover-btn--${variant} ${className}`;

  // Wujud Polimorfis 1: Render sebagai Link navigasi Router jika prop 'to' didefinisikan
  if (to) {
    return (
      <Link to={to} className={combinedClass} onClick={onClick} {...props}>
        {innerContent}
      </Link>
    );
  }

  // Wujud Polimorfis 2: Render sebagai elemen button standar untuk aksi event klik
  return (
    <button type="button" className={combinedClass} onClick={onClick} {...props}>
      {innerContent}
    </button>
  );
}
```

KESIMPULAN KETERCAPAIAN UNIT 07

Seluruh sub-judul pada unit 'Mengimplementasikan Pemrograman Berorientasi Objek (OOP)' telah terpenuhi melalui pewarisan kelas ApiError, enkapsulasi data, polimorfisme komponen UI, dan pemodelan DTO.